

Expense Management Tool for an IT Consultancy Company

Bachelor's Degree in Informatics Engineering
Specialization in Software Engineering

Bachelor's Thesis



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona



Natalia Yike Wang Yang

Thesis Supervisor

Sergio Alejandro Lobig Machado (FSP Consulting Services Limited)

Thesis Director

Marc Oriol Hilari (Department of Software Engineering)

Universitat Politècnica de Catalunya
Facultat Informàtica de Barcelona
Computer Science Department

May 19, 2026

Contents

1	Context	1
1.1	Introduction	1
1.2	Key Concepts	1
1.3	Problems to Solve	2
1.4	Proposed Solution	2
1.5	Stakeholders	3
1.5.1	Product Owner	3
1.5.2	Thesis Director	3
1.5.3	Development Team	3
1.5.4	FSP's Barcelona Team	3
1.5.5	Finance Department	3
1.5.6	IT Administrators	3
2	Justification	4
3	Scope	5
3.1	Objectives	5
3.2	Requirements	5
3.2.1	Functional Requirements	5
3.2.2	Non-Functional Requirements	6
3.3	Obstacles and Risks	7
3.3.1	Coding Bugs	7
3.3.2	Tight Project Deadlines	8
3.3.3	Lack of Experience in Certain Technologies	8
3.3.4	Company's Licenses and Security Permissions	8
4	Methodology	9
4.1	Development Methodology	9
4.2	Monitoring Tools	9
4.3	Validation Methods	10
5	Temporal Planning	11
5.1	Task Description	11
5.1.1	Project Starting	11
5.1.2	Project Management	11
5.1.3	Final Project Delivery	12
5.1.4	Design	12
5.1.5	Development	13
5.1.6	Frontend and Backend Development	13
5.1.7	Backend Development	15

	5.1.8	Frontend Development	16
5.2		Resources	16
	5.2.1	Human Resources	16
	5.2.2	Physical Resources	16
5.3		Summary of Tasks and Resources	17
5.4		Gantt Diagram	18
5.5		Risk Management	21
6		Budget and Sustainability	23
6.1		Budget	23
	6.1.1	Personnel Cost	23
	6.1.2	Generic Cost	24
	6.1.3	Other costs	26
	6.1.4	Total cost	27
	6.1.5	Management Control	27
6.2		Sustainability	28
	6.2.1	Economic Impact	28
	6.2.2	Environmental Impact	29
	6.2.3	Social Impact	30
7		Project Monitoring	31
7.1		Methodology	31
7.2		Temporal Planning	31
7.3		Budget	31
8		Alternative Analysis	32
8.1		Platform Selection	32
8.2		Data Extraction Selection	32
8.3		Architectural Pattern Selection	33
9		Knowledge Integration	34
10		Regulatory Identification	36
	10.1	General Data Protection Regulation (GDPR)	36
	10.2	Intellectual property (LPI)	37
		References	38

List of Figures

5.4.1 Gantt diagram.	20
------------------------------	----

List of Tables

5.2.1 Human resources.	16
5.2.2 Hardware resources.	16
5.2.3 Software resources.	17
5.3.1 Summary of project tasks with effort, dependencies, and resources	18
5.5.1 Summary of identified risks, their probabilities, affected tasks, and contingency resources.	22
6.1.1 Annual and hourly salaries for different project roles	23
6.1.2 Personnel cost breakdown by task, role, and total personnel cost	24
6.1.3 Amortization cost of the project's hardware resources	25
6.1.4 Electricity cost of the project's hardware resources	25
6.1.5 Azure resource cost	26
6.1.6 Total generic cost	26
6.1.7 Incidental cost.	27
6.1.8 Total cost of the project.	27

1 Context

This is a Bachelor's Thesis within the Software Engineering specialization of the Computer Engineering Degree, provided by the Barcelona School of Informatics at the Polytechnic University of Catalonia.

The project corresponds to modality B, as it is developed in a professional context in collaboration with FSP Consulting Services Limited. Consequently, this thesis is supervised under a dual-mentorship structure involving Marc Oriol Hilari, a lecturer in the specialization, and Sergio Alejandro Lobig Machado, a Senior Cloud Engineer at FSP.

1.1 Introduction

Organizations often hold company events that employees are required to attend. Travel to these events inevitably results in employees incurring various expenditures, such as flights, accommodation, and meals. As long as these expenses are business-related and not for personal use, companies typically offer reimbursement.

This is a common scenario within FSP's Barcelona team. FSP is a UK-founded private technology consultancy that has established Barcelona as one of its several office locations. As expected, the Barcelona-based team frequently travels to the UK for corporate events, meetings, and collaborative work, often struggling with high expenditures along the way.

The current method for managing these expense claims relies on a manual and document-based workflow. To do that, Barcelona's team must gather all tickets, record them in Excel spreadsheets, and email them to the Finance Department for processing and approval.

1.2 Key Concepts

The goal of this thesis is to apply and demonstrate the knowledge and competencies acquired throughout the author's Computer Engineering Degree. As a result, this document may contain jargon that could be challenging for readers without a background in this field. To facilitate reading of this thesis, the description of technical terms in this documentation is listed below and can be consulted as needed.

Expense Report Workflow

The full process of reporting an expense includes capturing the receipt, uploading the file, extracting information, submitting, and approving or rejecting the report.

Full-stack Application

An application that is composed of two parts, namely the frontend (visual user interface) and the backend (application's internal logic).

Pipeline

A set of ordered automated tasks executed sequentially.

Bottlenecks

Network traffic congestion.

Cloud Computing Platform

A virtual service that provides software development resources, replacing traditional physical resources.

RESTful API

API is the communication system between the frontend and backend, while REST (Representational State Transfer) is a widely recognized architectural pattern for designing APIs. REST standardizes API usage through uniform conventions, making it intuitive and straightforward for developers to understand and manage.

1.3 Problems to Solve

The existing hand-operated expense reporting system at FSP introduces inefficiencies for both employees and the Finance department. Specifically, employees spend excessive time entering detailed receipt information in Excel spreadsheets. Concurrently, the Finance Department undertakes labour-intensive work in verifying outdated spreadsheets.

Moreover, from FSP's perspective, this manual data entry is prone to human error. According to the company's historical data, approximately 30% of submitted reports require manual correction due to transcription errors or missing information. Consequently, this not only increases the risk of inaccuracy but also results in expenses being delivered to the wrong person.

Quantitatively, FSP's Barcelona team of 25 employees departs for the UK 2-4 times annually, generating an average of 8 receipts per person per trip. Each employee spends about 20 minutes reporting a single expense, while the Finance Department (4 people) takes 20 minutes to review the full set of expenses for each employee per trip. Collectively, this unproductive task accounts for roughly 225 lost hours per year and prolongs reimbursement cycles by about 18 days.

Therefore, the time consumed in this burdensome activity could be considerably reduced, enabling highly skilled employees to focus on activities that deliver higher value to the company. Hence, there is a need to optimize FSP's personnel investment.

1.4 Proposed Solution

FSP clearly lacks a more intelligent, automated expense management strategy to prevent employee frustration and guarantee accurate reimbursement of business expenses. This project aims to develop an automated web application that allows employees to manage, report, and submit expenses seamlessly on a single platform.

This application streamlines the entire workflow by offering effortless receipt uploads, AI-automated data extraction, and efficient expense report verification and approval. The primary goal is to simplify the previous expenses management workflow, optimizing time and effort for all parties, reducing human error, and ensuring information is transmitted securely and efficiently to the appropriate approver.

1.5 Stakeholders

With the problem and solution established, all parties involved in this project (also known as stakeholders) are identified below.

1.5.1 Product Owner

The Product Owner is responsible for understanding the client's interests and communicating them effectively to the application development team. Their primary role is to maximize the value of the final product from the client's perspective. They define the product's vision and requirements, ensuring that the application's features remain aligned with client needs throughout the development lifecycle.

In this case, Sergio Alejandro Lobig Machado serves as Product Owner, as he is the only FSP representative on this project.

1.5.2 Thesis Director

The Bachelor's Thesis Director oversees the project from an academic perspective. Marc Oriol Hilari fulfills this role, continuously monitoring progress to ensure that all required technical and transversal competencies are achieved by the end of the project.

1.5.3 Development Team

The development team is responsible for the full implementation of the application—from initial pipeline setup and database design to final testing and product deployment. For now, this role is carried out solely by the thesis author. However, this does not rule out the possibility of extending and upgrading the project in the future, potentially adding new developers to the team.

1.5.4 FSP's Barcelona Team

This application addresses FSP's Barcelona Team needs; as a consequence, they become the primary beneficiary of the product and are the application's end-users.

1.5.5 Finance Department

This group refers to the FSP's Finance department, which is responsible for processing the employee's reimbursement once the expense report is approved.

1.5.6 IT Administrators

IT administrators review and verify users' logs for security purposes and solve any technical problems. This collective is identified in FSP's IT department.

2 Justification

Given the project's focus on developing an expense management tool for FSP (Section 1.5), it is necessary to justify the project's execution. This section helps define the project's scope, objectives, and requirements, which are discussed in Section 3.

Although the current market offers numerous off-the-shelf solutions, they impose significant architectural limitations. Adopting a third-party platform implies conforming to its predefined AI model, functional workflow, and limited integration points. Since FSP is a technology consultancy, it is not seeking a solution that simply mirrors market offerings, but rather a more agile, customized approach.

Consequently, an in-house, scalable expense management application is preferred at FSP because it provides three key architectural advantages over commercial products:

- **Horizontal Scalability**

The application is built on a modern, cloud-native architecture based on decoupled microservices. This design allows each component to scale independently as the company grows. Scalability is often a major weak point of external providers, since it is beyond their control.

- **Maintainability and Extensibility**

The application codebase is owned by FSP and follows Clean Architecture principles by separating technical and business logic. This facilitates extension with new features and adaptation to changing business needs. As a result, long-term maintenance becomes painless, while FSP gains full control over the application lifecycle.

- **Easy Integration**

By implementing a decoupled, multi-layered Clean Architecture, the logic for interacting with any external system is encapsulated behind abstractions. The concrete implementations of these abstractions reside in a completely separate layer. Additionally, external systems are registered via Dependency Injection to adhere to the Dependency Inversion Principle. As a result, external services can be easily integrated without requiring critical changes in the backend API.

At the same time, this project represents the author's first independent application development. Consequently, it provides a valuable opportunity to demonstrate key Software Engineering competencies in a real-world context. In short, it is an ideal case that bridges the author's academic background and professional career.

3 Scope

As stated in the previous section, the main goal of this project is to **develop an in-house expense management web application that is scalable, maintainable, and integration-friendly**. To better achieve this objective, this section provides a more detailed breakdown of the project goal, specifies the application's functional and non-functional requirements, and concretizes the potential obstacles and risks.

3.1 Objectives

For a clearer understanding of the project scope and easier evaluation of its completion, the project's objective is broken down into smaller, more manageable sub-objectives:

SO1. Design the application's technology stack and the database structure.

SO2. Develop a full-stack web application.

SO3. Design a scalable pipeline to automate AI-driven data extraction from expense uploads.

SO4. Use a cloud infrastructure platform to manage external application resources.

SO5. Maintain a clean, reusable, maintainable, and integration-friendly application codebase.

3.2 Requirements

This section introduces the application's requirements, which are defined based on the project proposal document drafted by the Product Owner.

Following a Software Engineering approach, requirements are classified into two main groups: functional and non-functional. The former defines what features the application provides, while the latter describes how those features are delivered [1].

3.2.1 Functional Requirements

Due to the constrained thesis timeline, the application's functional requirements are prioritized by applying the MoSCoW methodology. MoSCoW categorizes requirements into four priorities: **Must have**, **Should have**, **Could have**, and **Won't have**.

Throughout the full classification process, the Product Owner serves as the primary reference point for determining the relevance of each requirement. The resulting MoSCoW-prioritized requirements list appears below.

By following the MoSCoW strategy, high-priority requirements are addressed first, while lower-priority ones are deferred in case of setbacks. Consequently, this enables early delivery of a high-value product; only requirements with negligible outcome impact can be omitted if time allocation falls short.

Before delving into the specific requirements, notice that a report is a set of AI-processed and user-reviewed expenses submitted collectively to the reviewer for approval or rejection of the corresponding total reimbursement amount.

Must have: requirements that are critical for the core application functionality.

FR1. The system must centralize expense data on a single platform.

FR2. The system must provide effortless receipt uploads and drag-and-drop.

FR3. The system must automate data extraction from receipts using an AI model.

FR4. The system must allow users to review and correct the AI-extracted data.

FR5. The system must enable simple report submissions to a predefined approver.

FR6. The system must require users to log in through their company's account.

FR7. The system must implement role-based access control with two roles: employees, with access to basic features, and admins, with access to additional features.

FR8. The system must allow admin users to approve or reject submitted reports.

FR9. The system must send email notifications to the report creator when the report is approved or rejected and to the administrator when new reports are pending review.

Should have: requirements that are relevant for the application functionality, but permit a certain delay in complete achievement.

FR10. The system should provide search bar and filters for uploaded expenses.

FR11. The system should allow users to group multiple expenses into a single report.

FR12. The system should display a history of submitted reports, categorized as: Pending Review, Approved, and Rejected.

FR13. The system should display to admin users a history of reviewed reports.

Could have: requirements that enhance the application usability and are fulfilled if time permits.

FR14. The system could allow users to edit report information.

FR15. The system could provide a search bar to find specific reports.

FR16. The system could allow admins to set expense budget limits for employees under their supervision.

Won't have: requirements that are irrelevant and will not be included in the application. As they fall outside the application's scope, these requirements are not explicitly defined.

3.2.2 Non-Functional Requirements

The subsequent requirements fall under the non-functional category. A brief description and acceptance criteria (where possible) are provided in each requirement for final project validation.

NFR1. The application must provide real-time updates on expense processing status.

The UI must update the expense statuses in less than **1 second** once the AI-driven data extraction is completed.

NFR2. The application must offer a responsive, user-friendly UI.

The UI must be **responsive**, adapting to any device or screen size [2], and user-friendly, allowing new users to upload, review, and save an expense within **5 minutes**.

NFR3. The application must ensure high performance and availability.

Upon user login, the application should load the previously uploaded expenses within **3 seconds**. Other basic user interactions (e.g., saving expense information, deleting expenses) should be completed under **500ms**. Additionally, the application should be available **24 hours** a day, **365 days** a year.

NFR4. The application must support scalability as the company grows.

The application must implement a decoupled architecture that separates the initial user request from any subsequent intensive processing workload. Additionally, its resources must be deployed on a cloud computing platform (or Platform-as-a-Service) that supports automatic horizontal scaling.

NFR5. The application must guarantee compliance with GDPR.

The acceptance criteria of this specific requirement are justified in Section 10.1.

NFR6. The application must enable a more efficient expense management process and a streamlined approval workflow.

The application must optimize the expense workflow from the current 20 minutes to **5 minutes** for both employees and finance staff, achieving a **75%** efficiency gain.

NFR7. The application must support easy integration with external systems.

The application must define abstractions for interactions with external systems, with their concrete implementations placed in a separate layer. The relationship between these abstractions and their implementations is established by adopting the Dependency Inversion Principle. This architectural approach ensures that future integrations of external components can be achieved with minimal code changes, thereby improving maintainability and extensibility.

NFR8. The project must support maintainability and extensibility.

The application must separate the technical implementation from the business logic, facilitating the extension of technical features or adaptation to a changing business structure.

3.3 Obstacles and Risks

Identifying obstacles and risks is crucial for more precise initial timeline estimation (further established in Section 5.4). Hence, this section outlines obstacles and risks that may hinder project execution. The analysis of their probability, impact and mitigation strategies is covered in detail in Section 5.5.

3.3.1 Coding Bugs

As the application development progresses, certain coding bugs will inevitably emerge, requiring additional time to fix. Given the high probability of encountering these risks, minor delays in the project are expected.

3.3.2 Tight Project Deadlines

The project timeline is constrained by the thesis deadline. Significant delays may affect the final product from fully meeting all defined requirements, representing a major challenge for the author.

3.3.3 Lack of Experience in Certain Technologies

The two previous situations are quite common in real-world engineering practice, and this one is no exception. Exploring and applying new technologies is a constant challenge to a software engineer's daily work.

3.3.4 Company's Licenses and Security Permissions

This project is executed within FSP's environment. The company's strategies to prevent cyberattacks may restrict the installation of new tools and resources. This supposes some constraints for the project, as certain installations are required throughout the development.

4 Methodology

ScrumBan is adopted throughout project execution. Selected monitoring tools support this methodology. Additionally, validation methods verify achievement of requirements defined in Section 3.2.

4.1 Development Methodology

This project applies the Agile methodology based on the ScrumBan framework. ScrumBan is a hybrid approach of Scrum and Kanban. Scrum features role assignments, focuses on team coordination, and aims to increment the product's value through predefined iterations (known as sprints). Kanban emphasizes flexibility with continuous workflows, incremental delivery, and strict work-in-progress (WIP) limits [3].

ScrumBan offers a wide range of options combining Scrum and Kanban methodologies, facilitating its adaptation to the project's context. The main roles defined for this project are a subset of Scrum roles: Product Owner and Development Team (their role descriptions and assignments are found in Section 1.5).

By contrast, the full development cycle reflects the Kanban continuous nature. Since the development team consists of a single developer, traditional Scrum ceremonies such as sprint planning, sprint review, and sprint retrospective are omitted. Instead, regular meetings with the Product Owner are scheduled every 2 days throughout the project to monitor the process.

Before starting the practical implementation, a product backlog (from Scrum) with all the project's tasks is defined to keep the project on track. Simultaneously, it is complemented by Kanban's pull principle, setting a **Work In Progress (WIP)** limit of **3**. WIP restricts the number of "in progress" tasks, preventing work overload and blocking next-steps tasks until any active task completes.

4.2 Monitoring Tools

The upcoming monitoring tools in this section are used to facilitate the application of Agile methodology.

Git

A cloud version control system that allows multiple developers to work on the same project within a shared repository, and enables a single user to develop several features at the same time. These capabilities make Git essential for the Kanban's continuous flow principle, allowing developers to proceed with new tasks when the current one is blocked.

In Git, a repository is the location where the application's source code is stored. It serves as a project space where developers submit their code, allowing changes to be tracked throughout the development process. Using a Git repository for source code version control provides high availability since it is cloud-based.

Azure DevOps

Azure is a cloud platform that offers multiple tools; among them, two relevant monitoring tools are used for this project: the Git repository (established earlier) and the backlog dashboard (for product backlog management).

In the backlog, four statuses are assigned to tasks:

- **New:** A task that is defined but is pending development, or a bug that is pending resolution.
- **Active:** A task that is currently in progress. As clarified in Section 4.1, there are up to 3 tasks in Active status, according to the WIP limit.
- **Resolved:** A bug that has been fixed.
- **Closed:** A task that has been developed, integrated, and tested.

Microsoft Teams

A communication tool used for conducting regular meetings with the Product Owner, which is mentioned earlier in Section 4.1.

4.3 Validation Methods

As mentioned in Section 4.2, the development code is managed in a Git repository. In this repository, new implementations are developed in their own branch (named Type/Author/Feature) and are merged into the main branch via Pull Requests (PRs). Each PR triggers the Continuous Integration (CI) and Continuous Delivery (CD) pipelines to compile, verify through rigorous testing, and build the new code.

In the CI/CD pipeline testing, both unit and integration tests are executed. Unit tests verify each component's behaviour functions as expected, while integration tests check that all components are properly connected. Additionally, unit tests must achieve at least 70% of code coverage to ensure effective testing.

Inside the code editor, Visual Studio Code extensions are installed from the beginning of code development. Before merging new code into the main branch, these extension tools must detect zero bugs and zero code smells (suspicious bug-prone code). Moreover, to demonstrate the time requirements established in NFR1 and NFR3 (Section 3.2.2), the browser's development tools will measure the application response times for each case.

Finally, to ensure application aligns with the established requirements (see Section 3.2), the product backlog tasks are not closed until the Product Owner confirms the specified requirements are met. To support this process, regular meetings with the Product Owner (according to Section 4.1) are held to monitor development progress. At the same time, the thesis director will supervise the project to ensure compliance with academic standards.

5 Temporal Planning

This section introduces the initial **122-day** project plan, scheduled from February 18, 2026, to June 19, 2026. The total estimated effort for all defined tasks is **513 hours**.

According to Polytechnic University of Catalonia, the bachelor's thesis carries a workload of 18 ECTS credits, equivalent to 540 effort hours. The discrepancy between the total estimated effort of 513 hours and the expected 540 hours results in a built-in buffer of **27 hours** to accommodate potential setbacks.

5.1 Task Description

The key to achieving effective time planning lies in a thorough analysis of all tasks, their priorities, and durations. This step ensures proper task sequencing and resource allocation.

Before this analysis, all tasks must be identified and categorized. For this reason, this section introduces the project tasks and groups them into: project starting, project management, final project delivery, design, development, and frontend/backend development

5.1.1 Project Starting

A solid project start requires a preliminary study and initial project definition. This aligns the project vision among all involved members and establishes a unified understanding of the project's scope and objectives. To achieve this, the subsequent tasks are defined:

PS1. Context and Scope

This task is divided into smaller sub-tasks:

- Analyze the current problem and identify all stakeholders.
- Define and propose a technical solution.
- Justify the project execution.
- Define the project scope in terms of objectives, requirements, obstacles, and risks.
- Select a methodology to guide project execution.
- Select the monitoring and validation tools to be used in the project.

PS2. Temporal Planning

Identify all tasks and resources, design an initial Gantt diagram timeline, and define a risk management strategy.

PS3. Budget and Sustainability

Define a project budget and analyze the sustainability viability of the proposed solution.

PS4. Initial Documentation

Consolidate the previous documentation into a definitive initial documentation.

5.1.2 Project Management

During this stage, the project's progress is monitored to keep it on track.

PM1. Product Owner Meetings

Run regular meetings with the Product Owner to discuss the development progress, implementation priorities, and any issues.

PM2. Thesis Director Review Meetings

Meet with the thesis director every two weeks to ensure the project is progressing correctly.

PM3. Product Backlog Update

Refine, update, and maintain the product backlog.

5.1.3 Final Project Delivery

Tasks for completing product delivery are listed below.

FPD1. User Manual Documentation

Write a user manual for the application's end-users as a reference guide.

FPD2. Final Thesis Documentation

Write the bachelor's thesis final documentation intended for an academic audience.

FPD3. Final Demonstration Video

Record a final demonstration of the application once it is ready for use.

FPD4. Bachelor's Thesis Defense

Plan and prepare a final exposition to defend the bachelor's thesis.

5.1.4 Design

This preliminary design phase establishes a clear application architecture to guide the subsequent development phase.

DT1. Use Case Diagram

Analyze all users' interactions with the system and design a Use Case Diagram that illustrates all actors, use cases, and relationships between use cases.

DT2. Conceptual Model Diagram

Design the UML database diagram, clarifying all database entities, their attributes, and the relationships between them.

DT3. Technology Stack Decision

Select the technology to be used for each application layer and provide a clear technology selection documentation that serves as a guide for the application development phase.

DT4. Logical and Physical Architecture Design

Design the logical and physical application architecture. The former describes how the implementation code is structured, including the purpose and role of each application layer. The latter explains how the application connects to external services and resources.

DT5. UI mockups Design

Deliver the mockup designs for each application page.

DT6. Security Design

Provide a security architecture design for user authentication, information submission and storage, and access control.

5.1.5 Development

Essential tasks for the application development are specified in this task group.

Dev0. Self-learning and study of unfamiliar technologies

Study and learn new application technologies in advance.

Dev1. Environment Configuration

Set up a ready-to-use environment before starting application development: install Visual Studio Code and essential extensions, create a Git repository, and perform other initial configuration tasks.

Dev2. Product Deployment

Release the final product for end-user availability.

Dev3. CI/CD Pipeline Implementation

Implement Continuous Integration and Continuous Delivery pipeline to automate compilation, unit and integration testing, and building of every updated code version.

Dev4. Logging and Monitoring

Set up logging and monitoring tools to track application performance and review logs.

Dev5. Git Repository Documentation

Maintain clear repository documentation for a technical audience, including an application setup guide, API contracts documentation, and meaningful code comments.

5.1.6 Frontend and Backend Development

As a full-stack web application, the implementation is divided into two main areas. The frontend is responsible for the user interface (UI) and user experience (UX). The backend implements the complex internal logic hidden from the frontend via a RESTful API and integrates with external services.

This development stage requires both frontend and backend development. Since it covers a wide range of functionalities, it is divided into two subgroups: Expense and Report Development, and User Access Development.

Expense and Report Development

This development subgroup includes the application's core functionalities, namely, the expense and report management.

ER1. Expense Uploads

Develop a file submission feature where the frontend handles the expense drag-and-drop and upload interface, and the backend registers the uploaded expense in the database.

ER2. Uploaded Expenses Real-time Dashboard

Develop a visual dashboard to display all uploaded expenses with their real-time processing status, which can be:

- **Processing:** The expense is pending processing by the AI model.
- **Pending Review:** The AI model has successfully analyzed the expense; the uploader can now review the AI-extracted information.
- **Completed:** The user has reviewed and completed all required information. The expense is ready to be included in a new or existing report.
- **Failed:** The AI couldn't extract data from the receipt (due to unavailability or unrecognized format). The uploader should manually fill in all required expense details.

ER3. AI-Extracted Expense Data Review and Correction

Develop a form page for users to review, edit, and save AI-extracted expense information, while updating the database with the corrected data.

ER4. Uploaded Expenses Deletion

Develop the expense deletion feature to delete one or multiple expenses at once, regardless of their current status. After user confirmation, they are marked as deleted in the database.

ER5. Expense Reports View, Creation, Edit, Submission, and Deletion

Develop report management functionalities (create, edit, submit, and delete reports) and the required RESTful API for synchronizing the database with the frontend changes.

User Access Development

The tasks described in this subgroup are contingent upon user authentication implementation before their execution can begin. It is important to note that administrators can set spending limits and manage and view submitted reports solely from employees under their supervision.

UA1. Company Email User Authentication

Develop the login feature that allows users to authenticate in the application through the company's email account, including:

1. User access verification.
2. (a) New account creation on first login.
(b) Existing user data recovery for returning users.

UA2. Role-based User Access

Develop the user role assignment (based on two main roles: regular employee or administrator) and expose to the user only their corresponding role's functionalities.

UA3. Submitted Report Approval and Rejection

Develop a feature to allow administrators to review, approve, or reject submitted reports, and update the database accordingly.

UA4. Employee Spending Limit Configuration

Develop an admin-only interface to set employees' spending limits and record the changes in the database.

UA5. Submitted and Reviewed Expenses Records Visualization

Develop a dashboard that displays records of users' submitted reports and administrators' reviewed reports.

UA6. Email Notification Service Integration

Develop the email notification service to notify users when their submitted reports are reviewed, and to administrators when new reports are pending for their review.

5.1.7 Backend Development

The following tasks are implemented exclusively in the application backend logic.

BD1. RESTful API Backend Logic Implementation

Build a RESTful API that follows the technology stack defined in the DT3 task to manage backend responsibilities via frontend requests.

BD2. Azure SQL Database Integration

Integrate with Azure SQL Database using Entity Framework Core migrations to manage the application's data storage and configure the initial database according to the diagram designed in the DT2 task.

BD3. Azure Blob Storage Integration

Store original receipt files using Azure Blob Storage.

BD4. Microsoft Document Intelligence AI Model Integration

Extract the relevant information from expenses using the Microsoft Document Intelligence AI model.

BD5. Automated AI-driven Data Extraction Pipeline

Integrate Event Grid, Service Bus, and Azure Function to implement an automated data extraction workflow that prevents bottlenecks during multiple simultaneous uploads:

1. Save the uploaded receipt file in Blob Storage and associated data in SQL Database.
2. Detect the event using Azure Event Grid and route the file to the Service Bus queue.
3. Once the new expense is in the queue, trigger the Azure Function to process the receipt file using the integrated AI model (BD4 Task).

BD6. Azure SignalR Real-time Notification Service Integration

Integrate SignalR to configure a SignalR Hub that sends real-time expense status updates, eliminating manual page refreshing whenever the AI processes a new receipt file.

BD7. Microsoft Entra ID User Authentication Integration

Handle employee authentication through the company email account via Microsoft Entra ID.

5.1.8 Frontend Development

This category covers all tasks that require only frontend logic implementation.

FD1. Expenses Search and Filter

Develop visual tools that enable users to search and filter uploaded expenses.

FD2. Reports Search

Provide a search bar to search specific reports by name.

5.2 Resources

Following the analysis of all required tasks for successful project completion, all necessary resources must be identified. Resources are divided into two main groups: human resources and physical resources. All human resources are listed in Table 5.2.1, specifying the responsible individual for each role. The physical resources are further categorized into software and hardware resources, identified and described in Tables 5.2.2 and 5.2.3.

5.2.1 Human Resources

Resource ID	Role Name	Responsible Individual
PO	Product Owner	Sergio Alejandro Lobig Machado
TD	Thesis Director	Marc Oriol Hilari
GT	GEP Tutor	Paola Pinto Del Monte
DT	Development Team	Thesis author

Table 5.2.1: Human resources. [Own creation]

5.2.2 Physical Resources

Hardware Resources

Resource ID	Resource Name	Components
OS	Office Supplies	A table, a chair, electricity, internet, and office rental.
HW	Hardware Resources	A computer, two monitors, a mouse, and a keyboard.

Table 5.2.2: Hardware resources. [Own creation]

Software Resources

Resource ID	Resource Name	Description
GIT	Git	A version control tool for managing the source code repository.
AZR	Azure Resources	Cloud services and resources: SQL Database, Blob Storage, AI model, Event Grid, Azure Function, Service Bus, SignalR, EntraID, and Azure DevOps.
MCS	Microsoft Tools	Communication and coordination platforms: Microsoft Teams, Calendar, Outlook, and Word.
VSC	Visual Studio Code	Source code editor with extensions and debugging tools.
FIB	FIB Platforms	Platforms from Barcelona School of Informatics(FIB): Racó, Atenea, FIB secretary, bachelor's thesis official website guide.
GM	Gmail	Email communication with university entities.
WEB	Web Browsers	Research Browsers: Mozilla Firefox, Google Chrome, Microsoft Edge.
OVL	Overleaf LaTeX Editor	Thesis formatting editor for final thesis documentation.
OT	Online Tools	Diagram design tools: Edraw, Draw.io, and Gantt Project.

Table 5.2.3: Software resources. [Own creation]

5.3 Summary of Tasks and Resources

See Table 5.3.1 for a summary of all defined tasks, including their identifiers, estimated effort in hours, any Finish-Start (FS) dependencies with other tasks, and the required resources. The final-to-start dependency column indicates which task must be fully completed before the task in that row can begin.

ID	Name	Time (h)	Dependencies	Resources
PS	Project Starting	50		
PS1	Context and Scope	20	-	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS2	Temporal Planning	10	PS1	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL, OT
PS3	Budget and Sustainability	10	PS2	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PS4	Initial Documentation	10	PS3	PO, TD, GT, DT, OS, HW, MCS, VSC, FIB, GM, WEB, OVL
PM	Project Management	46.5		
PM1	Product Owner Meetings	30.5	PS2	PO, DT, OS, HW, AZR, MCS, VSC
PM2	Thesis Director Review Meetings	8	-	DT, TD, OS, HW, AZR, VSC, FIB, GM, WEB
PM3	Product Backlog Update	8	PS2	DT, PO, OS, HW, AZR, WEB
FD	Final Product Delivery	56.5		
FPD1	User Manual Documentation	2	PM5	DT, OS, HW, VSC, WEB
FPD2	Final Thesis Documentation	30.5	-	DT, TD, OS, HW, FIB, GM, WEB, OVL, OT
FPD3	Final Demonstration Video	4	FPD2	DT, OS, HW, WEB
FPD4	Bachelor's Thesis Defense	20	FPD2	DT, TD, OS, HW, FIB, GM, WEB
DT	Design	16		
DT1	Use Case Diagram	2	PS1	DT, OS, HW, OT, WEB, MCS
DT2	Conceptual Model Diagram	4	PS1	DT, OS, HW, OT, WEB
DT3	Technology Stack Decision	2	PS1	DT, OS, HW, WEB, MCS
DT4	Logical and Physical Architecture Design	4	PS1	DT, OS, HW, WEB, MCS
DT5	UI mockups Design	2	DT1	DT, OS, HW, OT, WEB

(Continued on next page)

(Continued from previous page)

ID	Name	Time (h)	Dependencies	Resources
DT6	Security Design	2	DT3, DT4	DT, OS, HW, WEB, MCS
Dev	Development	86		
Dev0	Self-learning and study of unfamiliar technologies	8	DT3	DT, OS, HW, WEB
Dev1	Environment Configuration	2	DT2, DT5, DT6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev2	Product Deployment	8	UA4, UA5, UA6	DT, OS, HW, GIT, AZR, VSC, WEB
Dev3	CI/CD Pipeline Implementation	20	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
Dev4	Logging and Monitoring	8	Dev2	DT, OS, HW, GIT, AZR, VSC, WEB
Dev5	Git Repository Documentation	40	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
Frontend and Backend Development		128		
ER	Expense and Report Development	64		
ER1	Expense Uploads	16	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
ER2	Uploaded Expenses Real-time Dashboard	4	ER1, BD6	DT, OS, HW, GIT, AZR, VSC, WEB
ER3	AI-Extracted Expense Data Review and Correction	20	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER4	Uploaded Expenses Deletion	4	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	ER3, ER4	DT, OS, HW, GIT, AZR, VSC, WEB
UA	User Access Development	64		
UA1	Company Email User Authentication	8	BD7	DT, OS, HW, GIT, AZR, VSC, WEB
UA2	Role-based User Access	16	UA1	DT, OS, HW, GIT, AZR, VSC, WEB
UA3	Submitted Report Approval and Rejection	8	ER5, UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA4	Employee Spending Limit Configuration	8	UA2	DT, OS, HW, GIT, AZR, VSC, WEB
UA5	Submitted and Reviewed Expenses Record Visualization	4	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
UA6	Email Notification Service Integration	20	UA3	DT, OS, HW, GIT, AZR, VSC, WEB
BD	Backend Development	120		
BD1	RESTful API Backend Logic Implementation	20	Dev1	DT, OS, HW, GIT, AZR, VSC, WEB
BD2	Azure SQL Database Integration and Manipulation	20	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
BD3	Azure Blob Storage Integration	8	BD2	DT, OS, HW, GIT, AZR, VSC, WEB
BD4	Microsoft Document Intelligence AI Model Integration	20	BD2, BD3	DT, OS, HW, GIT, AZR, VSC, WEB
BD5	Automated AI-driven Data Extraction Pipeline	20	BD4	DT, OS, HW, GIT, AZR, VSC, WEB
BD6	Azure SignalR Real-time Notification Service Integration	16	BD5	DT, OS, HW, GIT, AZR, VSC, WEB
BD7	Microsoft Entra ID User Authentication Integration	16	BD1	DT, OS, HW, GIT, AZR, VSC, WEB
FD	Frontend Development	10		
FD1	Expenses Search and Filter	8	ER2	DT, OS, HW, GIT, AZR, VSC, WEB
FD2	Reports Search	2	ER5	DT, OS, HW, GIT, AZR, VSC, WEB
Total		513		

Table 5.3.1: Summary of project tasks with effort, dependencies, and resources. [Own creation]

5.4 Gantt Diagram

A Gantt diagram is attached at the end of this section. Notice that the execution order aligns with the prioritized requirements defined in Section 3.2.1 based on MoSCoW criteria. Continuous arrows represent Finish-Start (FS) dependencies between tasks, while dashed arrows represent Finish-Finish (FF) dependencies. FS dependencies indicate that successor tasks cannot start until their predecessor tasks are completed. FF dependencies show that successor tasks cannot finish until their predecessor tasks are completed.

Referring to Diagram 5.4.1, the project timeline spans **122 days** with a **one-week built-in buffer**. Considering the 513 total estimated hours, this equates to approximately **4 hours of work per day**. Consequently, the weekly workload is 28 hours, matching the weekly effort allocated in the Gantt diagram. Hence, the task effort estimations from

Table 5.3.1 are translated into the Gantt diagram using this pattern (e.g., the ER1 task, which costs 20 hours, represents 4 days of work in the diagram).

According to Section 4.1, regular meetings with the Product Owner are arranged every two days. Considering each meeting takes around 30 minutes, a total of 30.5 hours is dedicated to the **PM1** task. These meetings should be represented iteratively throughout the project timeline. However, since the diagram tool does not support this behavior, it is represented as a single concurrent task.

Excluding PM1, several other tasks are represented as concurrent and run concurrently:

- **PM2** remains undetermined due to undefined thesis director meetings, representing iterative appointments throughout the project lifetime.
- **PM3** starts after **PS2** (temporal planning) is completed and overlaps with full application development, as the product backlog requires regular updates based on the project's real-time progress.
- **FPD2** runs throughout the project execution for final delivery consistency.
- **Dev5**, which depends on **Dev1** (beginning once the repository is created), spans most of the project duration to satisfy objective SO5 from Section 3.1, as clean code requires ongoing, extensive documentation.

In addition, **ER1**, **ER3**, **UA2**, and **UA6** tasks have an initial effort estimation of 8 hours. However, the additional time allocated serves as a buffer for likely challenges. **ER1** is the first coding task; **ER3** requires extensive coordination with frontend and backend; **UA6** marks the author's first experience implementing email notifications; **UA2** requires a prior database record of the employee hierarchy. Finally, **Backend Development** tasks similarly incur additional time costs.

Expenses Reporting Tool for an IT Consultancy

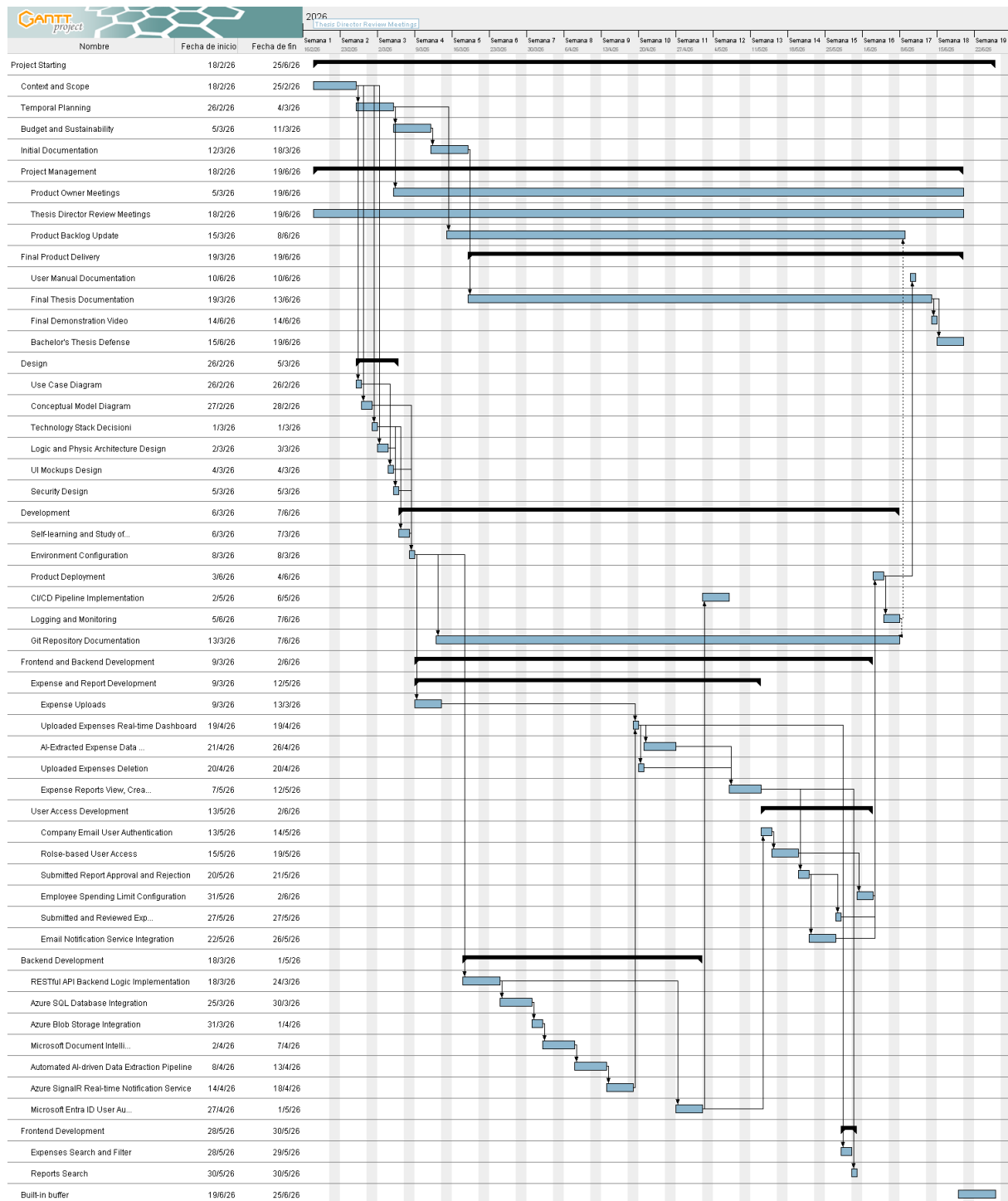


Figure 5.4.1: Gantt diagram. [Own creation]

5.5 Risk Management

Possible risks affecting project progress are outlined in Section 3.3. With the timeline now established, contingency plans are defined to mitigate potential obstacles. This section presents each identified risk along with its probability, impact, and corresponding mitigation strategy. Moreover, all identified risks, along with the corresponding affected tasks and the contingency resources required for mitigation, are detailed in Table 5.5.1.

Coding Bugs

Probability: Medium

Impact: High

Coding bugs are inevitable, making thorough testing essential. As outlined in Section 4.3, every code change undergoes rigorous testing through automated CI/CD pipelines, including unit and integration tests. During the Dev1 task, ESLint and SonarQube Visual Studio Code extensions are installed to automate bug detection from the very beginning of the implementation.

If bugs persist, additional debugging time is allocated, using up to **10 hours** of the buffer time. It is important to fix critical bugs before proceeding with new feature development to prioritize functional application delivery.

However, thanks to the installed extensions and consistent testing, the number of bugs is minimized, reducing the likelihood of significant setbacks. Furthermore, complex tasks already include built-in buffer time, as mentioned in section 5.4, and the reserve hours are available to cover any delays that may arise.

Tight Project Deadlines

Probability: High

Impact: Medium

Software development projects often struggle with tight deadlines. Due to the author's limited professional experience, deviation between the actual task effort and the initial time estimation is anticipated. However, since this project follows Agile methodology, iterative meetings help to detect and manage overruns at an early stage.

To overcome this challenge, **9 hours** of overhead serve as a lifesaver. This is an intentional allocation, as the project is scheduled to finish several days before the thesis defense period. Therefore, a one-week delay is permitted without affecting the final submission.

In the worst-case scenario, if the project misses the expected deadline, a selection of non-essential features, based on the MoSCoW principles defined in Section 3.2.1, will be completed only if time permits. Since the Gantt diagram's (Section 5.4) tasks follow MoSCoW prioritization, this is a viable solution. This alternative ensures a high-value solution is delivered to FSP but may be adopted solely with the Product Owner's approval.

Lack of Experience in Certain Technologies**Probability:** High**Impact:** Medium

Given that several technologies used in application implementation lie beyond the author's expertise, mitigation plans have been established to address this challenge. Time for learning and becoming familiar with these technologies is factored into the project timeline (Task Dev0), thereby minimizing potential setbacks.

Since a prior study cannot fully mitigate this risk, a proposed solution is to set aside up to **8 hours** from the available buffer. If delays are prolonged, assistance from the FSP team serves as a fallback. Seeking timely assistance is an essential professional skill, as it helps prevent unnecessary deadline extensions.

Company's Licenses and Security Permissions**Probability:** Medium**Impact:** Low

Another risk involves FSP's security system. Cyberattacks are taken more seriously every day, often requiring developers to request and wait for permission to install resources and extensions. Consequently, these restrictions may limit initial resource availability.

To prevent this blocker, Dev1 has been created to install all the previously identified tools and extensions in advance. However, this prior setup does not remove the probability of installing forthcoming tools.

As a contingency, a reordering of the project backlog can be considered using the Gantt chart (Figure 5.4.1) as a reference. Tasks with no dependencies, such as BD7 and the subsequent UA category tasks, may be prioritized and initialized earlier. Likewise, other non-dependent tasks (e.g., Dev5 task) may also be advanced to maintain project progress.

Risk	Probability	Time (h)	Affected Tasks	Resources
Coding Bugs	15%	10	Dev3, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Tight Project Deadlines	30%	9	Dev2-4, ER1-5, UA1-6, BD1-7, FD1-2	DT, HW
Lack of Experience in Certain Technologies	30%	8	Dev2, Dev3, ER1-5, UA1-6, BD1-7,	DT, HW
Company's Licenses and Security Permissions	15%	0	Dev2, BD1-7	DT, HW

Table 5.5.1: Summary of identified risks, their probabilities, affected tasks, and contingency duration and resources. [Own creation]

6 Budget and Sustainability

Before closing the initial project documentation, a detailed analysis of its economic viability and sustainability across economic, environmental, and social dimensions is provided. The budget is based on tasks and resources defined in Section 5, while the sustainability assessment evaluates the long-term impact of the proposed solution.

6.1 Budget

This section details all project expenses, justifying each cost and its corresponding calculation. The budget is structured into four main categories: Personnel Cost (PC), Generic Cost (GC), Contingency Cost (CC), and Incidental Cost (IC).

6.1.1 Personnel Cost

To obtain a realistic personnel cost estimate, the average hourly wage for each personnel role was calculated in advance using market salary data[4], assuming 260 working days per year and an average of 7.5 working hours per day.

Table 6.1.1 specifies the hourly wage calculation for each role, including the Social Security (SS) cost. Each human resource identified in Section 5.2.1 is assigned to a specific role in the table, resulting in 5 different roles:

- **Project Manager (PM)**: Responsible for planning and leading the entire project.
- **Junior Full-Stack Developer (D)**: Responsible for the full application implementation
- **QA Tester (T)**: Responsible for verifying the application functionalities.
- **Software Architect (SA)**: Responsible for designing the application’s technical architecture while considering the project requirements and scope.
- **Technical Writer (TW)**: Responsible for drafting clear, accurate project documentation.

Role	Annual Salary (€)	Total including SS (€)	Cost / Hour (€)	Assigned HR
Project Manager (PM)	43,300	56,290	28.87	PO, TD, GT
Full-stack Junior Developer (D)	29,800	38,740	19.87	DT
QA Tester (T)	26,888	34,954.4	17.93	DT
Software Architect (SA)	41,800	54,340	27.87	DT
Technical Writer (TW)	31,300	40,690	20.87	DT

Table 6.1.1: Annual and hourly salaries for different project roles. [Own creation]

Given each personnel member’s hourly wage and their corresponding effort hours in each task (extracted from Table 5.3.1), the total personnel cost is calculated as shown in Table 6.1.2. Each task cost results from multiplying both values.

ID	Name	Total hours	Hours per role						Cost (€)
			PM	D	T	SA	TW		
PS	Project Starting	50	80	0	0	0	0	1,433.33	
PS1	Context and Scope	20	20	0	0	0	0	577.33	
PS2	Temporal Planning	10	10	0	0	0	0	288.67	
PS3	Budget and Sustainability	10	10	0	0	0	0	288.67	
PS4	Initial Documentation	10	10	0	0	0	0	288.67	
PM	Project Management	46.5	38.5	46.5	0	0	0	2,035.17	
PM1	Product Owner Meetings	30.5	30.5	30.5	0	0	0	1486.37	
PM2	Thesis Director Review Meetings	8	8	8	0	0	0	389.87	
PM3	Product Backlog Update	8	0	8	0	0	0	158.93	
DT	Design	16	0	0	0	16	0	445.87	

(Continued on next page)

(Continued from previous page)

ID	Name	Total hours	PM	D	T	SA	TW	Cost (€)
DT1	Use Case Diagram	2	0	0	0	2	0	55.73
DT2	Conceptual Model Diagram	4	0	0	0	4	0	111.47
DT3	Technology Stack Decision	2	0	0	0	2	0	55.73
DT4	Logical and Physical Architecture Design	4	0	0	0	4	0	111.47
DT5	UI mockups Design	2	0	0	0	2	0	55.73
DT6	Security Design	2	0	0	0	2	0	55.73
FPD	Final Product Delivery	56.5	0	5	0	0	51.5	1173.97
FPD1	User Manual Documentation	2	0	1	0	0	1	40.73
FPD2	Final Thesis Documentation	30.5	0	0	0	0	30.5	636.43
FPD3	Final Demonstration Video	4	0	4	0	0	0	79.47
FPD4	Bachelor's Thesis Defense	20	0	0	0	0	20	417.33
Dev	Development	86	0	76	10	0	0	1689.12
Dev0	Self-learning and study of unfamiliar technologies	8	0	8	0	0	0	158.93
Dev1	Environment Configuration	2	0	2	0	0	0	39.73
Dev2	Product Deployment	8	0	8	0	0	0	158.93
Dev3	CI/CD Pipeline Implementation	20	0	10	10	0	0	377.92
Dev4	Logging and Monitoring	8	0	8	0	0	0	158.93
Dev5	Git Repository Documentation	40	0	40	0	0	0	794.67
	Frontend and Backend Development	128	0	128	0	0	0	2,542.93
ER	Expense and Report Development	64	0	64	0	0	0	1,271.47
ER1	Expense Uploads	16	0	16	0	0	0	317.87
ER2	Uploaded Expenses Real-time Dashboard	4	0	4	0	0	0	79.47
ER3	AI-Extracted Expense Data Review and Correction	20	0	20	0	0	0	397.33
ER4	Uploaded Expenses Deletion	4	0	4	0	0	0	79.47
ER5	Expense Reports View, Creation, Edit, Submission, and Deletion	20	0	20	0	0	0	397.33
UA	User Access Development	64	0	64	0	0	0	1,271.47
UA1	Company Email User Authentication	8	0	8	0	0	0	158.93
UA2	Role-based User Access	16	0	16	0	0	0	317.87
UA3	Submitted Report Approval and Rejection	8	0	8	0	0	0	158.93
UA4	Employee Spending Limit Configuration	8	0	8	0	0	0	158.93
UA5	Submitted and Reviewed Expenses Record Visualization	4	0	4	0	0	0	79.47
UA6	Email Notification Service Integration	20	0	20	0	0	0	397.33
BD	Backend Development	120	0	120	0	0	0	2,384.00
BD1	RESTful API Backend Logic Implementation	20	0	20	0	0	0	397.33
BD2	Azure SQL Database Integration and Manipulation	20	0	20	0	0	0	397.33
BD3	Azure Blob Storage Integration	8	0	8	0	0	0	158.93
BD4	Microsoft Document Intelligence AI Model Integration	20	0	20	0	0	0	397.33
BD5	Automated AI-driven Data Extraction Pipeline	20	0	20	0	0	0	397.33
BD6	Azure SignalR Real-time Notification Service Integration	16	0	16	0	0	0	317.87
BD7	Microsoft Entra ID User Authentication Integration	16	0	16	0	0	0	317.87
FD	Frontend Development	10	0	10	0	0	0	198.67
FD1	Expenses Search and Filter	8	0	8	0	0	0	158.93
FD2	Reports Search	2	0	2	0	0	0	39.73
Total		513	88.5	385.5	10	16	51.5	11,913.05

Table 6.1.2: Personnel cost breakdown by task, role, and total personnel cost. [Own creation]

6.1.2 Generic Cost

This category covers the project operational costs and is based on the resources identified in Section 5.2.2. The project duration, calculated in advance in Section 5 (513 hours over 122 days), is used in each cost calculation.

Amortization

The amortization cost corresponds to the proportion of hardware resource usage time relative to their total usable lifespan. Hence, this cost is formulated as:

$$\text{Amortization (€)} = \frac{\text{Resource Price}}{\text{Total Depreciation Hours}} \times \text{Project Usage Hours}$$

The usage hours equal the project execution time, since hardware resources are used

throughout the project. Assuming a four-year depreciation period and the same annual working hours from PC (260 working days * 7.5 working hours/day), which is a total of 1950 hours, the following equation is applied for each hardware resource in Table 6.1.3:

$$\text{Amortization (€)} = \text{Resource price} \times \frac{1}{4 \text{ years}} \times \frac{1 \text{ year}}{1950 \text{ working hours}} \times \text{hours used}$$

Hardware Resources	Price (€)	Quantity	Total Price (€)	Usage Time (h)	Amortization (€)
PC Dell 16 Pro [5]	1,193.53	1	1,193.53	513	78.50
Wireless Mouse [6]	11.99	1	16.99	513	1.12
Dell Monitor [7]	171.82	2	343.64	513	22.60
Keyboard [8]	19.99	1	19.99	513	1.31
Total					102.22

Table 6.1.3: Amortization cost of the project's hardware resources. [Own creation]

Electric cost

Only hardware resources require electricity and are the only resources considered in this cost. Electricity costs are calculated by multiplying the current electricity cost of **0.1748 €/kWh** [9] by the energy consumed and the resource usage time. Table 6.1.4 details the calculations and the resulting total cost:

Hardware Resources	Power (W)	Usage Time (h)	Energy Consumed (kWh)	Electricity Cost (€)
PC Dell 16 Pro [5]	65	513	33.345	5.83
Wireless Mouse [6]	0.05	513	0.02565	0.00
Dell Monitor [7]	24	513	12.312	2.15
Keyboard [8]	0.1	513	0.0513	0.01
Total			45.73	7.99

Table 6.1.4: Electricity cost of the project's hardware resources. [Own creation]

Internet cost

The basic internet plan costs €21.95 per month [10]; hence, this cost is calculated as follows:

$$\text{Internet cost} = 21.95 \text{ €/month} \times \frac{1 \text{ month}}{30 \text{ days}} \times 122 \text{ days} = \mathbf{€89.26}$$

Office rental cost

The project development always takes place in the FSP office. The FSP office is located in Plaça Francesc Macià, Barcelona, and demands a monthly rental fee of €6,000. Given that there is enough space to accommodate up to 25 employees (which is the number of employees at FSP's Barcelona team, as defined in Section 1.3), this is the cost calculation:

$$\text{Office rental cost} = 6,000 \text{ €/month} \times \frac{1 \text{ month}}{30 \text{ days}} \times \frac{1 \text{ office}}{25 \text{ employees}} \times 122 \text{ days} = \mathbf{€976.00}$$

Software resources cost

Most software resources are open-source and are available for free. The primary expenses in this cost are Azure and Microsoft resources.

Azure charges a pay-as-you-go fee and offers a customized price for each resource usage [11]. Since the application target user group is small, the most basic/free tier for each

resource is considered. A monthly fee of €3.96 is estimated for this project, as shown in Table 6.1.5.

Azure resources (€/month)	Cost
SQL DataBase	3.90
Bandwidth	0.01
Blob Storage	0.01
Log Analytics	0.01
Event Grid	0.01
Service Bus	0.01
Functions	0.01
Total	3.96

Table 6.1.5: Azure resource cost. [Own creation]

Accordingly, the total Azure resources cost in the project is derived:

$$\text{Azure resource cost} = 3.96 \text{ €/month} \times \frac{1 \text{ month}}{30 \text{ days}} \times 122 \text{ days} = \mathbf{\text{€24.16}}$$

Based on the Microsoft annual fee of €99 [12], the corresponding total cost is the following:

$$\text{Microsoft resource cost} = 99 \text{ €/year} \times \frac{1 \text{ year}}{365 \text{ days}} \times 122 \text{ days} = \mathbf{\text{€33.09}}$$

Thus, the total software resources cost equals the sum of both costs:

$$\text{€24.16} + \text{€33.09} = \mathbf{\text{€57.25}}$$

Total generic cost

By summing all previously calculated costs, the total generic cost is obtained, as shown in Table 6.1.6.

Concept	Cost (€)
Amortization	102.22
Electricity	7.99
Internet	89.26
Office Rental	976.00
Software	57.25
Generic Cost (GC)	1,232.72

Table 6.1.6: Total generic cost. [Own creation]

6.1.3 Other costs

Contingencies

No project strategy can eliminate the possibility of encountering obstacles. Therefore, a time buffer (27 hours) has been included to cover potential setbacks. However, simply adding extra time is insufficient, as it also entails additional resource consumption.

For this reason, a portion of the project budget must be allocated for these potential scenarios. Therefore, the contingency cost amounts to **€1,314.57**, which is the 10% of the total general cost calculated so far:

$$PC (\text{€}) + GC (\text{€}) = 11,913.05 + 1,232.72 = \mathbf{13,145.77 \text{ €}}$$

Incidental Costs

Incidental costs encompass the project risks analysis outlined in Section 5.5. The cost of each risk is calculated by extracting its probability, time cost, and required resources from Table 5.5.1. A breakdown calculation is presented in Table 6.1.7. Note that the resources required for each risk are identified as Rx (R1, R2, ...).

Incident	Risk (%)	Time (h)	R1	R1 Cost (€/h)	R2	R2 Cost (€/h)	Total Cost (€)	Risk Cost (€)
Coding Bugs	15	10	D	19.87	HW	1.99	200.66	30.10
Tight Project Deadlines	30	9	D	19.87	HW	1.79	180.59	54.18
Lack of Experience in Certain Technologies	30	8	D	19.87	HW	1.59	160.53	48.16
Company's Licenses and Security Permissions	15	0	D	19.87	HW	0.00	0.00	0.00
Total								132.44

Table 6.1.7: Incidental cost. [Own creation]

6.1.4 Total cost

After summing the total costs from each section, the final project budget is determined, as shown in Table 6.1.8.

Concept	Cost (€)
Personnel Cost (PC)	11,913.05
Generic Cost (GC)	1,232.72
Contingency Cost (CC)	1314.57
Incidental Cost (IC)	132.44
Total Budget	14,592.78

Table 6.1.8: Total cost of the project. [Own creation]

6.1.5 Management Control

Challenges, both expected and unexpected, are always present in any project. Even with a precisely estimated timeline and budget, it is very common for a project to fall short of the initial plan.

At the end of the project, the real execution cost is calculated using the same methodology as the initial budget estimation. Based on these results, the project budget deviation is determined across three areas:

- **Human resources deviation:** To contrast the actual effort with the previous estimated effort of human resources in each task.

$$\text{Human resources deviation} = \sum_{i \in pit} (\text{estimated_cost_per_hour}_i - \text{real_cost_per_hour}_i) \times \text{total_real_hours}_i$$

Where i indexes each individual in pit , the set of all personnel involved in that task.

- **Amortization deviation:** To compare the previously estimated use of hardware resources with the actual usage time of those resources.

$$\text{Amortization deviation} = \sum_{i \in hr} (\text{estimated_usage_hours}_i - \text{real_usage_hours}_i) \times \text{price_per_hour}_i$$

Where i indexes each resource in hr , the set of all hardware resources used in the project.

- **Total cost deviation:** To determine each task's real cost deviation from the estimated cost. The general cost described in the formula represents the sum of the personnel and generic costs, excluding contingency and incident costs. Hence, the following formula is applied to each task:

$$\text{Total cost deviation} = (\text{estimated_general_cost} - \text{real_general_cost})$$

Note that the resulting total cost deviation value can be either positive or negative. A positive value indicates an overestimation of the initial budget for that resource or task. In this case, the additional budget can be reallocated to address issues that arise during the project. Conversely, a negative value indicates an underestimation, requiring contingency and incident costs to cover the budget shortfall.

Furthermore, some costs are not included in the deviation calculation. For example, Azure offers a fixed monthly fee; therefore, overusage will not affect the overall cost deviation. The same applies to the remaining omitted costs.

6.2 Sustainability

The technological revolution has accelerated dramatically in recent decades. A clear example is the launch of the first telephone (1876), the first computer (1937), the first laptop (1974), the first smartphone (2007), and the first artificial intelligence (2017) [13].

This acceleration pace reveals that the time it takes for a technological invention to emerge and significantly impact society continues to shrink. Consequently, this exponential growth underscores the increasing importance of evaluating the economic, environmental, and social impacts of technological projects. These three dimensions together constitute sustainability, the topic discussed in this section.

Building on this trend, technology will continue shaping our future in the coming years. The primary mission of engineers is to make people's daily lives easier. However, developing a solution is not enough; a study of its sustainability effects is also a must.

Therefore, this section addresses sustainability questions, enabling a better evaluation of the project's impact across the economy, environment, and society. This analysis raises awareness of the weight of sustainability in project development.

Personally, the author recognizes limited sustainability knowledge, mainly covering environmental footprint and social impact. The economic dimension and multiple sustainability indicators require further study. The subsequent subsection answers a series of key sustainability questions within the author's current expertise.

6.2.1 Economic Impact

Regarding PPP: Have you estimated the project execution cost?

The project budget is €14,592.78, as detailed in Section 6.1. This corresponds to the full "bill" for the application, including all personnel, hardware amortization, software licensing, and contingency costs.

Regarding Useful Life: How are currently solved economic issues (costs...) related to the problem that you want to address (state of the art)?

According to the problems stated in Section 1.3, the current time-consuming, low-efficiency expense management process delivers minimal value while misallocating company personnel costs for both employees and finance staff.

Regarding Useful Life: How will your solution improve economic issues (costs, etc.) with respect to other existing solutions?

The primary benefit of the proposed solution is the operational efficiency optimization. The current manual process requires approximately 225 hours per year, as estimated in Section 1.3. The application is designed to reduce the time required for this activity by at least 75%, in accordance with the non-functional requirement NFR6 from Section 3.2.2.

Based on this reduction and the actual personnel wages at FSP, the solution generates annual cost savings of approximately €5,062.5. Given the estimated development cost of €14,592.78, the return on investment (ROI) is projected to be achieved within 3 years. This demonstrates that the proposed solution is more economically viable than investing in external solutions, supporting its adoption as a sustainable long-term approach.

However, an initial budget misestimation could extend the ROI period. This is mitigated with the 27-hour built-in buffer (calculated in Section 5) and the MoSCoW prioritization strategy (employed in Section 3.2.1). As a result, a high-value product is most likely to be delivered, since *Could have* requirements can be postponed in case of budget shortfalls.

6.2.2 Environmental Impact**Regarding PPP: Have you estimated the project's environmental impact?**

The only resource directly affecting the environmental footprint is energy consumption. As shown in Table 6.1.4, the total energy consumption is 45.73 kWh. Using a conversion factor of 0.283 kg CO₂/kWh [14], the project's carbon footprint is around 12.94 kg CO₂.

Regarding PPP: Have you considered minimizing its impact, for example, by reusing resources?

The project's environmental impact is minimized by conducting the full project development in FSP's shared office, reusing existing lighting and preventing unnecessary energy consumption.

Regarding Useful Life: How is the problem that you want to address currently solved (state of the art)?

The current employees' annual effort spent on manual expense management is 225 hours, corresponding to an estimated energy consumption of 12.63 kWh (assuming a PC power consumption of 65 W). As the proposed solution reduces the required effort by 75%, the associated energy consumption is expected to decrease by around 3.16 kWh. In addition, digitalizing the process reduces the environmental impact associated with printing, transportation, and the physical handling of expense records.

Moreover, the application uses Azure for software resources. The Azure resources required for application development cost less than traditional physical resources. This cost optimization results from a significant reduction in resource consumption. At the same time, cloud resources do not require physical transportation or hardware replacement, further reducing electronic waste.

Nevertheless, suboptimal code may lead to excessive CPU and memory consumption, resulting in a negative environmental impact on the project. Therefore, adherence to high-quality coding practices is emphasized throughout the project development. As a

result, a more sustainable application is delivered, using only essential resources and minimizing overall consumption throughout its lifecycle.

Regarding Useful Life: How will your solution improve the environment with respect to other existing solutions?

Developing an internal application tailored to the company's scale is inherently more resource-efficient than relying on external parties, since only useful functionalities are implemented.

6.2.3 Social Impact

Regarding PPP: What do you think you will achieve—in terms of personal growth—from doing this project?

This project enables the author to apply prior academic knowledge and software development skills in a real-world context. As a result, it enriches their working experience as future software engineer. Furthermore, it is the author's first experience managing and leading an entire project independently, strengthening self-learning, self-management skills and the ability to build software projects from scratch.

Regarding Useful Life: How is the problem that you want to address currently solved (state of the art)?

The current expense management process is handled manually through traditional methods. This involves manually entering data into Excel spreadsheets, emailing them to the finance department, and reviewing them line by line.

Regarding Useful Life: How will your solution improve the quality of life (social dimension) with respect to other existing solutions?

The proposed solution optimizes time spent on expense management while reducing employees' frustration and fatigue. This frees employees from this tedious task, allowing additional time to focus on more meaningful work. As a result, it maximizes employees' productivity.

However, negative effects are also encountered with the proposed application. An initial resistance to change and a steep learning curve are expected. In addition, a non-intuitive UI would exacerbate the adoption process. To mitigate the last risk, the Product Owner is actively involved in task DT5, a user manual guide is required as part of task PM4 (both defined in Section 5.1), and compliance with NFR2 from Section 3.2.2 is ensured.

Another potential risk is excessive reliance on AI extraction, leading employees to poorly review the AI-extracted data, increasing the risk of errors. To mitigate this, the workflow is redesigned so that reports cannot be submitted until employees have explicitly reviewed the extracted information.

Regarding Useful Life: Is there a real need for the project?

Regarding all project sustainability impacts (social, environmental, and economic), there is a clear need to build a robust solution. This solution overcomes challenges from all involved parties while introducing a sustainability-focused approach.

7 Project Monitoring

As the project reaches its mid-stage, the adopted methodology is analyzed, and the current project status is compared with the initial timeline. As a result, minor adjustments are made to the established methodology and schedule if necessary.

7.1 Methodology

The ScrumBan adoption for this project has been key to its successful execution so far. By combining selected characteristics of both Scrum and Kanban, this approach is well-suited to the project's needs.

Moving away from traditional Scrum ceremonies to brief meetings every two days has been a beneficial decision for the project. This approach enables continuous project monitoring and discards traditional ceremonies that provide low-value in a single-developer context. At the same time, restricting meeting duration to 30 minutes helps focus on critical issues and prevents overruns.

Regarding the WIP limit, it is not a restrictive constraint that blocks development progress. Quite the opposite, it has been a useful approach. Limiting the number of active tasks reduced the development boundary, allowing a more dynamic project flow.

Thanks to this methodology, the project's actual execution is kindly adhering to the scheduled timeline. At this stage, most of the tasks due today have been completed. Hence, it is not required to reassess the selected methodology.

7.2 Temporal Planning

Despite the project's overall progress being consistent with the initial plan, minor adjustments have been applied as expected. A small reordering of tasks was needed to keep progress moving forward.

The main change from the initial Gantt diagram is the postponement of the Dev3 task, due to the author's limited experience with pipeline development. A one-week dedication to this task revealed a shortfall compared to the original estimate of five days. Even though the effort exceeded the planned allocation, the task is still not completed. Given that it is not a high-priority requirement and that it has no strong dependencies on other tasks, Dev3 has been rescheduled to the final development stage.

However, all other planned tasks have met their expected deadlines so far, and the delay of the Dev3 task does not have a significant impact on the project. The buffer time set aside for anticipated deviations is being used to complete this task and to prevent it from becoming a bottleneck in the project's execution.

7.3 Budget

The initial budget estimation remains aligned with the project's current progress. So far, no significant deviations have been identified, and the planned costs continue to reflect the current project progress. As a result, the budget is well-founded and does not currently require any adjustments. At this stage, the financial plan can be considered stable and appropriate for the remaining development effort.

8 Alternative Analysis

During the application definition phase, a set of possible alternatives was evaluated. Next, an analysis of these options, including the selected alternative and its justification, is presented.

8.1 Platform Selection

Currently, there are three main application platforms: web applications, native mobile applications, and hybrid mobile applications. Web applications are accessible from any device through a browser by entering the URL. Native mobile applications are installed on a mobile phone and are specific to a single operating system, either Android or iOS. Hybrid mobile applications combine the characteristics of both web and native platforms.

A native application requires deep specialization for both operating systems, demanding higher development costs. Its adoption may also be slower, as users must install the application before using it. While mobile phones are convenient for uploading the expenses via the camera, performing the subsequent AI-extraction review and report creation is much more comfortable on a larger screen.

Conversely, a web application offers more practical advantages. A web application can be accessed from any device without installation. Development is much faster, a single codebase implementation is more than enough, because it is valid for all users. Ongoing maintenance and updates are instant after simply publishing the new version on the server.

Given the primary use case of creating expense reports at a desk and the need for universal accessibility without installation barriers, the **web platform** is selected. In terms of cost and development effort, this one is also the most suitable option.

8.2 Data Extraction Selection

After uploading the expenses, their relevant information can be extracted in several ways: manually entering the data, using OCR technology, or leveraging a pre-trained AI service. FSP's traditional expense management approach relies on manual data entry, which sometimes can be frustrating, error-prone, and time-consuming (as discussed in Section 1.3).

OCR is a technology that captures text from images and converts it into machine-readable text. However, this technology introduces certain drawbacks. It is sensitive to low-quality images, increasing the probability of errors. It also follows strict rules; even minor differences in keyword capitalization or data formatting may produce inconsistencies. Consequently, the solution must account for countless receipt formats, which raises the need for regular maintenance.

Unlike OCR, AI models not only read text but also understand context across a wide variety of formats without relying on rigid rules. With this capability, they can also recognize structured data such as line items and their corresponding amounts, going well beyond what conventional text-scanning technologies can achieve.

While traditional OCR with extensive rules is a possibility, it is overly brittle and requires

intensive maintenance to handle the wide variety of real-world receipts. An **AI model** is a more effective choice, since it provides higher accuracy and significantly lower development and maintenance overhead. In addition, it fulfills FR3, one of the core project requirements outlined in Section 3.2.1.

8.3 Architectural Pattern Selection

The automated data extraction pipeline, defined in the BD5 task in Section 5.1, can be implemented using two distinct architectural patterns. The first is a synchronous monolithic architecture, in which the entire process, from the initial upload to the AI processing of the expense, is handled through a single API call. The second is an asynchronous microservices-based architecture, where each service is responsible for a specific function in the workflow.

An expense upload triggers the POST `/api/expenses` call. A simple POST request is quick and lightweight, whereas the AI-powered extraction introduces considerable processing delay. Consequently, the two architectural patterns exhibit opposite behaviours.

In the case of synchronous monolithic architecture, the POST call cannot complete until AI extraction is finished, so the server response time is considerably prolonged. This is particularly harmful in bulk-upload scenarios, where expenses are processed sequentially, degrading user experience and reducing server availability. At the same time, reliability is low: any error in the end-to-end workflow causes the entire request to fail, requiring users to retry the upload.

In an asynchronous microservices-based architecture, the POST call is fully decoupled from the AI extraction process. Single uploads complete instantly, and bulk uploads are handled concurrently, preventing server overload. After a successful expense upload, the expense is routed to a queue and processed by the AI when resources are available. In addition, a more robust error-handling strategy is applied: if the AI model fails to extract the data, the expense is preserved and sent back to the queue for a later retry.

Due to its negative impact on user experience and scalability, the synchronous monolithic approach is discarded, biasing the design toward the alternative solution. The **asynchronous architecture with microservices** delivers immediate feedback to the user, prevents API bottlenecks, and results in a more resilient and scalable system. By making this choice, the non-functional requirements NFR1, NFR3, and NFR4 (Section 3.2.2) are also fulfilled.

9 Knowledge Integration

Since this bachelor's thesis belongs to the software engineering specialization, the knowledge acquired in various specialization subjects is applied throughout the project lifecycle. Below is a list of each of these subjects, along with their specific contributions to this project

Web Services and Applications (ASW)

Provided that the project is centered on a web application, this is undoubtedly the first subject to address. The key outcome of this subject is a solid understanding of RESTful API design. At the same time, hands-on experience with Ruby on Rails has strengthened the author's skills in database management and migrations. Finally, the introduction of the Swagger tool has also improved API testing and documentation. As a result, these concepts have been actively applied throughout the project.

Software Engineering Projects (PES)

PES is also a key component of this project. This subject gave the author the first opportunity to develop a software application from the ground up, covering all necessary phases of the software lifecycle. Through this experience, the subject highlights that software engineering goes beyond coding, incorporating essential planning and management activities to ensure project success.

On top of that, it enabled the practical application of Agile methodologies, which fostered pragmatic habits such as continuous documentation and regular updates to the product backlog. Additionally, it provided the opportunity to build a decoupled frontend-backend application and work with unfamiliar frameworks, significantly strengthening the author's expertise in React.

The introduction of new tools has also been beneficial for the author and has therefore enabled their application in this project. For example, editor extensions that support coding best practices and Git version control.

Software Project Management (GPS)

GPS is the first subject that introduced the author to agile methodology concepts, one of the most relevant approaches in the current software market. It also taught how to create an initial timeline and budget estimate. Furthermore, it covered other common software project concepts, such as risk analysis and the Gantt diagram.

Requirements Engineer (ER)

The project's context study, requirements analysis, and use case diagrams are all documented in accordance with the concepts introduced in this specific subject.

Software Engineering Introduction (IES) and Software Architecture (AS)

These two subjects introduced the basics for building UML diagrams and designing layered architecture.

Database (BD) and Database Design (DBD)

As the name suggests, these subjects helped the author design a secure and optimized database, write efficient queries, and become familiar with SQL databases.

Cybersecurity Management (GCS)

The theory presented in this subject raised the author's awareness of the wide variety of cyberattack strategies and underscored the importance of preventing any foreseeable cybersecurity risks.

Informatics' Social and Environmental Aspects (ASMI)

While the previous subjects provided more technical knowledge, this one focuses on non-technical aspects. Specifically, it has significantly enriched the project's sustainability analysis.

10 Regulatory Identification

Aside from satisfying the client's interests, the application's legal framework must also be taken into account. To this end, all regulations that the application must comply with are listed, along with a justification for compliance or non-compliance.

10.1 General Data Protection Regulation (GDPR)

This application is targeted at FSP's Barcelona office team. As part of European citizenship, this group is legally protected by the GDPR. Therefore, the application must comply with all applicable GDPR provisions in this project.

According to the data minimization principle (**Article 5(1)(c)**), the application must gather and store strictly essential information for the stated purpose. Any information that falls outside this scope must not be collected.

In this case, the application aims to manage employees' expenses. Hence, the application only stores relevant data to approve or reject the reported expenses. In addition, the collected user personal information is limited to the user's name and email, which are used exclusively for the email notification service.

Pursuant to **Article 6 of the GDPR**, any data processing must rest on a valid legal basis and pursue a lawful purpose. In this project, the data processing is based on contractual necessity, since the company expense reimbursement is a condition for fulfilling the employee's employment contract.

Following **Article 32 of the GDPR**, technical and organizational measures are required to guarantee data security. This is adequately addressed by applying several techniques. Secure communication between the client and the server is achieved using HTTPS, which encrypts data in transit. Data at rest is stored in Azure SQL Database and encrypted through Transparent Data Encryption, and receipt files are stored in Azure Blob Storage with private access.

Finally, to fully comply with **Article 32**, the application access is protected by Microsoft Entra ID. User authorization is managed by App Roles, restricting each user's access to their own expenses and each administrator's access to the expenses corresponding to their own team. This last feature ensures compliance with the principle of least privilege.

10.2 Intellectual property (LPI)

This final degree project is also governed by the Intellectual Property Law (LPI) and the UPC regulations. As stated in **Article 10.1 of LPI**, the software application developed in this project, including source code, diagrams, and documentation, is considered a work protected by copyright.

The LPI distinguishes between moral rights and economic rights. Moral rights, such as the right to be credited, are inalienable. Economic rights, on the other hand, are proprietary and can be assigned to third parties.

Given that this bachelor's thesis is under B modality (in collaboration with FSP), the UPC regulations approved in 2022 (**Acord CG/2022/05/23**) are also applied. As a consequence, the student retains moral rights, primarily the right to be credited, while the ownership of the intellectual property and the application's economic rights are assigned to FSP. This has been previously agreed upon by both parties, UPC and the company.

Moreover, the application's use of third-party software complies with their respective licenses. Permissive licenses such as the MIT license are used for React, .NET, and many other libraries. In addition, Azure services are used under Microsoft's terms of service. These licenses are compatible within a commercial application context.

References

- [1] Jun Yin Zhou. *Functional and Non-Functional Requirements*. Concordia University, 2004. URL: <https://spectrum.library.concordia.ca/id/eprint/7991/1/MQ91163.pdf>.
- [2] Parlakkiliç A. “Evaluating the effects of responsive design on the usability of academic websites.” In: *Education and information technologies*, 27(1), 1307-1322 (2022). URL: <https://doi.org/10.1007/s10639-021-10650-9>.
- [3] M. Alqudah and R. Razali. “An Empirical Study of Scrumban Formation based on the Selection of Scrum and Kanban Practices”. In: *Int. J. Adv. Sci. Eng. Inf. Technol.* 8.6 (Dec. 2018), pp. 2315–2322.
- [4] *Salary in Spain*. Talent.com. 2026. URL: <https://es.talent.com/salary> (visited on 03/05/2026).
- [5] *Dell Pro 16 Laptop*. Dell. 2026. URL: <https://www.dell.com/es-es/shop/port%C3%A1tiles-de-dell/port%C3%A1til-dell-pro-16/spd/dell-pro-pc16250-laptop> (visited on 03/05/2026).
- [6] *Logitech M171 Wireless Mouse*. Logitech. 2026. URL: <https://www.logitech.com/es-es/shop/p/m171-wireless-mouse.910-006867> (visited on 03/05/2026).
- [7] *Dell Pro 24 Plus P2425H Monitor*. Dell. 2026. URL: <https://www.dell.com/es-es/shop/monitor-dell-pro-24-plus-p2425h/apd/210-bmff/monitores-y-accesorios> (visited on 03/05/2026).
- [8] *Logitech K120 USB Keyboard*. Logitech. 2026. URL: <https://www.logitech.com/es-es/shop/p/k120-usb-standard-computer> (visited on 03/05/2026).
- [9] *Electricity Price in Spain*. Tarifa Luz Hora. 2026. URL: <https://tarifaluzhora.es/> (visited on 03/05/2026).
- [10] *Orange Fiber Offer*. Orange. 2026. URL: <https://www.orange.es/ofertas-tarifas/clientes/oferta-fibra-adicional/> (visited on 03/05/2026).
- [11] *Azure Pricing*. Microsoft. 2026. URL: <https://azure.microsoft.com/en-us/pricing/> (visited on 03/05/2026).
- [12] *Microsoft 365 Pricing*. Microsoft. 2026. URL: <https://www.microsoft.com/es-es/microsoft-365/buy/compare-all-microsoft-365-products> (visited on 03/05/2026).
- [13] *History of Technology Timeline*. Encyclopaedia Britannica. 2026. URL: <https://www.britannica.com/story/history-of-technology-timeline> (visited on 03/05/2026).
- [14] *Nuevos factores de emisión*. ECODES. 2024. URL: <https://ecodes.org/hacemos/cambio-climatico/mitigacion/ceroco2/nuevos-factores-de-emision-2024-lo-que-necesitas-saber-para-calcular-tu-huella-con-rigor> (visited on 03/17/2026).